Efficient Time Series Forecasting Using TinyML Models

Muskan Agrawal[1], Dr. Rahul Kumar Verma[1]

[1]Department of Computer Science and Artificial Intelligence, Indian Institute of Information Technology Lucknow

***Abstract*—Time-series classification on resource-constrained edge devices has become increasingly critical with the rapid growth of IoT applications. While numerous machine learning approaches exist, the impact of preprocessing techniques on both accuracy and computational efficiency in TinyML settings remains insufficiently studied. This paper investigates the effect of filtering and downsampling strategies on three benchmark human activity recognition datasets—UCI-HAR, PAMAP2, and WISDM—using lightweight models including Logistic Regression, Decision Trees, and a compact 1D Convolutional Neural Network (TinyCNN). Multiple preprocessing pipelines combining Butterworth filtering, Savitzky–Golay smoothing, naive downsampling, and mean downsampling were systematically designed and evaluated. Performance was assessed using a composite score that balances classification accuracy with inference latency, reflecting real-world edge deployment constraints. Experimental results show that TinyCNN achieves the highest classification accuracy across all datasets, while Decision Trees provide near-zero inference times suitable for ultra-low-power environments. The findings demonstrate that appropriate preprocessing can significantly improve efficiency with minimal accuracy trade-off, offering practical guidance for selecting preprocessing strategies in TinyML-based HAR deployments.**

***Index Terms*—Downsampling, Edge Computing, Filtering, Human Activity Recognition, Logistic Regression, Signal Preprocessing, Time Series Classification, TinyML**

I. Introduction

THE rapid growth of Internet of Things (IoT) devices and wearable sensors has generated an unprecedented volume of time-series data, particularly in the domain of Human Activity Recognition (HAR). With over 75 billion IoT devices now deployed globally, the demand for intelligent, on-device data processing has never been greater [1]. Efficiently processing such data demands machine learning models that achieve high classification accuracy while remaining computationally feasible for deployment on resource-constrained hardware [1].

Traditional approaches to machine learning inference rely on cloud-based architectures, where raw sensor data is transmitted to remote servers for processing. While effective, this paradigm introduces significant latency, raises data privacy concerns, and is heavily dependent on network connectivity — limitations that are unacceptable in many real-world HAR applications such as fall detection for elderly care, real-time gesture control, and industrial worker safety monitoring. These constraints have motivated a fundamental shift toward on-device intelligence.

TinyML addresses this challenge by enabling the deployment of machine learning models directly on microcontrollers and embedded systems with severely limited memory, processing power, and energy budgets. Unlike cloud-based inference, on-device execution supports real-time, low-latency, and privacy-preserving predictions, making TinyML particularly well-suited for applications such as wearable health monitoring, gesture recognition, and industrial IoT systems [2]. Typical TinyML target platforms include ARM Cortex-M class microcontrollers with as little as 256 KB of flash memory and 64 KB of SRAM, imposing strict constraints on model size and inference complexity.

Despite significant research efforts directed at optimizing model architectures for edge deployment, the role of data preprocessing remains comparatively underexplored [3]. Signal filtering and downsampling directly influence input signal quality, data dimensionality, and computational overhead, thereby affecting not only model accuracy but also training efficiency and inference latency — factors that are critical in TinyML contexts. In particular, the ordering of preprocessing operations (filtering before or after downsampling) and the choice of filter type have received little systematic investigation in the context of resource-constrained deployment.

Furthermore, most existing TinyML studies evaluate models on a single dataset, making it difficult to draw generalizable conclusions about the relationship between preprocessing strategies and model performance across varying data characteristics. Datasets differ significantly in feature dimensionality, sampling frequency, class distribution, and inter-subject variability — all of which interact with preprocessing in ways that are not yet well understood.

In this work, we systematically investigate the impact of preprocessing pipelines on TinyML-based time-series classification across three benchmark HAR datasets: UCI-HAR, PAMAP2, and WISDM. We evaluate combinations of filtering methods (Butterworth and Savitzky–Golay) and downsampling strategies (naive and mean), applied across five distinct pipeline configurations totalling 13 experimental setups. Three model classes are compared — Logistic Regression, Decision Trees, and a compact 1D Convolutional Neural Network (TinyCNN) — and their performance trade-offs are assessed using a composite metric that jointly considers classification accuracy and inference latency, reflecting practical edge deployment constraints. The main contributions of this work are as follows:

- A systematic cross-dataset evaluation of 13 preprocessing pipeline configurations for TinyML-based HAR classification.
- A composite scoring metric that jointly optimizes for accuracy and inference latency, designed to reflect real-

world edge deployment priorities.
- Empirical evidence that preprocessing pipeline design is as impactful as model selection for TinyML deployments, with dataset-dependent recommendations for practitioners.
- Comparative analysis of classical machine learning models and a lightweight CNN under varying preprocessing conditions, providing actionable guidance for resource-constrained deployment.

II. Literature Review

Human activity recognition using inertial measurement units (IMUs) has been extensively studied over the past two decades. Early approaches relied on hand-crafted features extracted from sliding windows of accelerometer and gyroscope data, feeding them into classical classifiers such as Support Vector Machines and k-Nearest Neighbors [4]. These methods required significant domain expertise to engineer discriminative features and struggled to generalize across subjects and environments.

The UCI-HAR dataset, introduced by Anguita et al. [5], became a benchmark for evaluating HAR models and demonstrated that frequency-domain features combined with linear classifiers could achieve over 96% accuracy. This work established the sliding window segmentation paradigm that remains standard in HAR research today. Subsequent work introduced deep learning approaches—particularly CNNs and LSTMs—that can learn hierarchical temporal features directly from raw sensor signals, eliminating the need for manual feature engineering [6].

The PAMAP2 dataset [7] extended HAR research to a broader range of physical activities including household tasks and sports, collected from subjects wearing multiple IMU sensors on the wrist, chest, and ankle. Its higher feature dimensionality (52 channels) and diverse activity set make it a more challenging and realistic benchmark compared to UCI-HAR. The WISDM dataset [8] provided smartphone-based accelerometer data with a focus on practical, real-world deployment conditions, capturing the high inter-subject variability typical of unconstrained environments.

In the TinyML domain, Banbury et al. [9] benchmarked various model architectures for keyword spotting and image classification on microcontrollers, highlighting that model size and inference latency are as important as accuracy. Their work introduced the concept of a multi-objective evaluation framework for edge ML, which directly motivates the composite scoring metric proposed in the present work. Work by Warden and Situnayake [1] popularized the TensorFlow Lite Micro framework, enabling deployment of compact CNN models on ARM Cortex-M microcontrollers and establishing practical guidelines for TinyML model design and quantization.

Several studies have explored lightweight model architectures specifically for HAR. Ordóñez and Roggen [3] proposed deep CNN and LSTM architectures for multimodal wearable HAR, demonstrating that end-to-end learned representations outperform hand-crafted features on complex multi-sensor datasets. However, these models are computationally expensive and unsuitable for direct deployment on microcontrollers without significant compression. This has motivated the use of simpler architectures such as shallow CNNs and classical classifiers in TinyML settings, where the primary constraint is inference latency rather than raw accuracy.

Signal preprocessing for HAR has received comparatively less systematic attention in the literature. Butterworth low-pass filters are commonly applied to remove high-frequency noise from accelerometer signals [10], and have been shown to improve classifier performance by attenuating motion artifacts. Savitzky–Golay filters, originally proposed for analytical chemistry applications [11], have found application in signal processing due to their ability to smooth data while preserving peak amplitudes — a property particularly useful for detecting sharp activity transitions in HAR. Downsampling is often applied to reduce data dimensionality and computational cost, but the interaction between downsampling factor, filter type, and operation ordering remains underexplored in the TinyML context [3].

A key gap in existing literature is the absence of a controlled, cross-dataset study that systematically isolates the effect of preprocessing pipeline design on both model accuracy and inference efficiency. Most prior work either fixes the preprocessing pipeline and varies the model, or evaluates preprocessing on a single dataset without considering deployment constraints. The present work directly addresses this gap by providing a comprehensive, cross-dataset comparison of preprocessing pipelines under a unified evaluation framework specifically optimized for TinyML deployment constraints.

III. Research Methods

A. Datasets

Three publicly available benchmark datasets for Human Activity Recognition (HAR) were selected to ensure a comprehensive and diverse evaluation. These datasets differ in feature dimensionality, sampling frequency, and complexity, enabling assessment of model robustness under varying real-world conditions.

UCI-HAR: The UCI Human Activity Recognition dataset [5] was collected from 30 subjects performing six daily activities—walking, walking upstairs, walking downstairs, sitting, standing, and laying—using a Samsung Galaxy S II smartphone mounted on the waist. Sensor data was recorded from a 3-axis accelerometer and 3-axis gyroscope at 50 Hz, yielding 9 input features per timestep. The dataset contains 10,299 windowed samples with an input shape of $(8239 \times 64 \times 9)$.

PAMAP2: The Physical Activity Monitoring dataset [7] was collected from 9 subjects performing 18 physical activities, including household tasks, Nordic walking, and cycling, using three IMU sensors placed on the wrist, chest, and ankle. With 52 features per timestep, PAMAP2 represents a higher-dimensional and more complex sensing scenario, with an input shape of $(2192 \times 64 \times 52)$.

WISDM: The Wireless Sensor Data Mining dataset [8] consists of 3-axis accelerometer data collected from 36 subjects via smartphone during six activities: walking, jogging, upstairs, downstairs, sitting, and standing. With only 3 features

per timestep and an input shape of $(38434 \times 64 \times 3)$, WISDM represents a lightweight, practical deployment scenario with high inter-subject variability.

B. Data Preparation

Raw sensor data from each dataset was processed into fixed-length sliding windows prior to model training. For PAMAP2, invalid activity entries (label = 0, corresponding to transitional or undefined states) were removed to ensure label integrity. Features were separated from labels by excluding the timestamp and activity columns, yielding a clean feature matrix $\mathbf{X}$ and label vector $\mathbf{y}$:

```
data = data[data[1] != 0]
X = data.drop(columns=[0, 1]).values
y = data[1].values
```

For all datasets, data was segmented into fixed windows of length 64 with appropriate overlap. Labels were encoded using integer encoding compatible with sparse categorical cross-entropy loss. Each dataset was split into training and test sets following the original benchmark partitions where available, ensuring comparability with prior work.

C. Preprocessing Pipelines

The design of the preprocessing framework constitutes a core contribution of this work. A controlled experimental setup was established in which identical models were trained across multiple preprocessing configurations to isolate the effect of signal transformation techniques on model performance. In total, 13 preprocessing pipelines were evaluated across five categories.

A. No Processing (Raw Baseline): Raw sensor data was used directly without any filtering or downsampling, serving as the reference configuration against which all other pipelines were compared.

B. Filtering Only (2 pipelines): Two smoothing filters were applied independently to the raw signal:

- *Butterworth Filter:* A 3rd-order low-pass Butterworth filter with a cutoff frequency of 5 Hz at a sampling rate of 50 Hz, applied to attenuate high-frequency noise while preserving the low-frequency activity signal.
- *Savitzky–Golay Filter:* A polynomial smoothing filter with window length 7 and polynomial order 2, applied to smooth signals while better preserving peak amplitudes and activity transitions.

C. Downsampling Only (2 pipelines): Two downsampling strategies were applied independently to reduce temporal resolution and computational cost:

- *Naive Downsampling:* Every alternate sample was retained (factor = 2), effectively halving the sequence length.
- *Mean Downsampling:* Consecutive pairs of samples were averaged, reducing sequence length while partially preserving signal energy.

D. Downsampling → Filtering (4 pipelines): Each of the two downsampling strategies was combined with each of the two filtering methods, applied in the order of downsampling first followed by filtering. This yields all pairwise combinations (PnC) of the two downsampling and two filtering methods, resulting in 4 pipelines:

- Naive Downsampling → Butterworth Filter
- Naive Downsampling → Savitzky–Golay Filter
- Mean Downsampling → Butterworth Filter
- Mean Downsampling → Savitzky–Golay Filter

In this ordering, the signal is first downsampled to reduce dimensionality, and filtering is subsequently applied to suppress any aliasing artifacts introduced during downsampling.

E. Filtering → Downsampling (4 pipelines): Similarly, each of the two filtering methods was combined with each of the two downsampling strategies, applied in the order of filtering first followed by downsampling. This again yields all pairwise combinations (PnC) of the two filtering and two downsampling methods, resulting in 4 pipelines:

- Butterworth Filter → Naive Downsampling
- Butterworth Filter → Mean Downsampling
- Savitzky–Golay Filter → Naive Downsampling
- Savitzky–Golay Filter → Mean Downsampling

In this ordering, noise is first removed through filtering to preserve signal quality, and downsampling is subsequently applied to reduce data dimensionality and computational overhead.

The rationale for evaluating both orderings in categories D and E is to determine whether the sequence of operations has a measurable impact on model performance — specifically, whether filtering before downsampling better preserves signal structure compared to downsampling first.

D. Models

Three model classes were selected to span the spectrum from classical machine learning to lightweight deep learning, enabling evaluation of accuracy–efficiency trade-offs across varying computational budgets.

Logistic Regression: A linear classifier trained with a maximum of 200 iterations (`max_iter=200`). Logistic Regression serves as a lightweight baseline, offering near-instantaneous inference and minimal memory footprint, representative of the simplest viable TinyML classifier.

Decision Tree: A non-linear, rule-based classifier with maximum depth 15 and minimum samples per split of 10 (`max_depth=15, min_samples_split=10`). Decision Trees offer good interpretability and deterministic inference, making them suitable for low-power microcontroller deployment.

TinyCNN: A compact 1D Convolutional Neural Network designed to capture local temporal dependencies in sensor signals while remaining deployable on edge hardware. The architecture consists of:

- `Conv1D(16, 3, activation='relu')`
- `MaxPooling1D(2)`
- `Conv1D(32, 3, activation='relu')`
- `MaxPooling1D(2)`
- `Conv1D(32, 3, activation='relu')`
- `Flatten()`
- `Dense(32, activation='relu')`

- `Dense(num_classes, activation='softmax')`

The model was compiled using the Adam optimizer with sparse categorical cross-entropy loss. We selected models spanning classical machine learning (Logistic Regression, Decision Tree) and deep learning (TinyCNN) to evaluate trade-offs between accuracy and computational efficiency across diverse resource constraints.

E. Evaluation Metrics

Each pipeline–model combination was evaluated across five metrics to ensure suitability for edge deployment where latency and computational constraints are critical:

- **Accuracy:** Classification accuracy on the held-out test set, serving as the primary performance indicator.
- **Training Time:** Total time (in seconds) required to train the model on the full training set.
- **Inference Time:** Total time (in seconds) to generate predictions on the entire test set.
- **Inference Time per Sample:** Per-sample inference latency, directly reflecting responsiveness on edge devices.
- **Composite Score:** A unified ranking metric defined as:

$$\text{Score} = \text{Accuracy} - 0.1 \times T_{\text{inf}} - 0.000001 \times T_{\text{train}} \quad (1)$$

where T_{inf} is the total inference time and T_{train} is the total training time. This formulation reflects real-world TinyML deployment priorities: inference latency is heavily penalized as edge devices require fast, responsive predictions; training time is lightly penalized as training occurs offline and is therefore less critical. Accuracy remains the primary optimization objective. Results for each model were ranked in descending order of composite score to identify the optimal preprocessing configuration per model class.

IV. Results and Discussion

This section presents the experimental results obtained across three benchmark HAR datasets — UCI-HAR, PAMAP2, and WISDM — for all 13 preprocessing pipelines evaluated on three model classes. Results are ranked by composite score to reflect real-world TinyML deployment priorities. For each model, the top 3 configurations are reported alongside the raw baseline to illustrate the improvement contributed by preprocessing. All experiments were conducted under identical conditions to ensure fair comparison.

A. UCI-HAR Dataset

Table I presents the top-ranked configurations for each model on the UCI-HAR dataset.

TinyCNN achieved the highest accuracy on UCI-HAR, with the top configuration — Naive + Savgol (Downsampling → Filtering) — reaching 95.44% accuracy and a composite score of 0.9313, compared to the raw baseline score of 0.9065. Notably, downsampling-first pipelines consistently outperformed filtering-first pipelines for TinyCNN on this dataset, suggesting that dimensionality reduction prior to smoothing better preserves discriminative temporal features for convolutional processing.

TABLE I
Top Results on UCI-HAR Dataset (Ranked by Score)

Model	Pipeline	Acc.	Train(s)	Inf.(s)	Score
TinyCNN	Naive+Savgol (DS→Filt.)	0.9544	3.546	0.230	0.9313
TinyCNN	Mean+Savgol (DS→Filt.)	0.9544	3.583	0.233	0.9310
TinyCNN	Naive (DS Only)	0.9529	3.487	0.231	0.9298
TinyCNN	Raw (Baseline)	0.9461	4.160	0.396	0.9065
DecisionTree	Mean+BW (DS→Filt.)	0.8641	5.513	0.003	0.8638
DecisionTree	Naive+BW (DS→Filt.)	0.8563	5.628	0.000	0.8556
DecisionTree	BW+Mean (Filt.→DS)	0.8558	5.289	0.003	0.8555
DecisionTree	Raw (Baseline)	0.8024	11.396	0.006	0.8018
LogReg	Raw (Baseline)	0.6117	4.681	0.004	0.6112
LogReg	Savitzky–Golay (Filt. Only)	0.6117	4.604	0.005	0.6111
LogReg	Savgol+Naive (Filt.→DS)	0.6092	2.580	0.003	0.6089
LogReg	Mean (DS Only)	0.6088	2.627	0.002	0.6085

Decision Trees performed competitively, achieving up to 86.41% accuracy with near-zero inference times (0.003s), making them highly attractive for latency-critical edge deployments. The raw baseline score of 0.8018 confirms that hybrid preprocessing pipelines provide meaningful improvement for tree-based models, with the best configuration gaining approximately 0.062 in composite score over the baseline.

Logistic Regression showed negligible sensitivity to preprocessing, with the raw baseline already achieving the joint-highest score of 0.6112. This plateau at approximately 61% accuracy regardless of preprocessing configuration indicates insufficient model capacity for the 6-class UCI-HAR problem, and confirms that preprocessing alone cannot compensate for fundamental model limitations.

B. PAMAP2 Dataset

Table II presents the top-ranked configurations for each model on the PAMAP2 dataset.

TABLE II
Top Results on PAMAP2 Dataset (Ranked by Score)

Model	Pipeline	Acc.	Train(s)	Inf.(s)	Score
TinyCNN	Savgol+Naive (Filt.→DS)	0.9454	3.050	0.266	0.9187
TinyCNN	Savitzky–Golay (Filt. Only)	0.9399	2.803	0.243	0.9156
TinyCNN	Savgol+Mean (Filt.→DS)	0.9417	2.914	0.282	0.9135
TinyCNN	Raw (Baseline)	0.9271	2.828	0.242	0.9029
LogReg	BW+Naive (Filt.→DS)	0.9326	0.961	0.003	0.9323
LogReg	Mean+BW (DS→Filt.)	0.9326	1.205	0.003	0.9323
LogReg	Mean+Savgol (DS→Filt.)	0.9308	1.044	0.003	0.9305
LogReg	Raw (Baseline)	0.8889	1.492	0.010	0.8879
DecisionTree	Naive+BW (DS→Filt.)	0.8579	8.919	0.009	0.8571
DecisionTree	Mean+BW (DS→Filt.)	0.8488	8.630	0.010	0.8479
DecisionTree	Butterworth (Filt. Only)	0.8087	16.748	0.023	0.8065
DecisionTree	Raw (Baseline)	0.7359	17.113	0.019	0.7340

PAMAP2 produced a notable reversal compared to UCI-HAR. Logistic Regression, despite being the simplest model, achieved composite scores competitive with TinyCNN — scoring up to 0.9323 against TinyCNN's best of 0.9187. This is attributable to PAMAP2's high feature dimensionality (52 channels), which provides sufficient linear separability for a logistic classifier, combined with Logistic Regression's dramatically lower inference time (0.003s vs. 0.266s for TinyCNN). Furthermore, both models showed substantial improvement over their respective raw baselines (TinyCNN: +0.016, LogReg: +0.044), confirming that preprocessing is particularly beneficial on this dataset. This finding highlights a critical

insight for TinyML deployment: on high-dimensional, well-structured datasets, simpler models can be more deployment-efficient without significant accuracy loss.

For TinyCNN, Savitzky–Golay based pipelines consistently ranked highest on PAMAP2, suggesting that the smoother, peak-preserving characteristics of the Savitzky–Golay filter are better suited to the richer, multi-sensor signals in PAMAP2 compared to the more aggressive attenuation of the Butterworth filter.

Decision Trees showed the largest absolute gain from preprocessing on PAMAP2, improving from a raw baseline score of 0.7340 to 0.8571 — a gain of 0.1231 — confirming that preprocessing is especially impactful for tree-based models on complex, high-dimensional data.

C. WISDM Dataset

Table III presents the top-ranked configurations for each model on the WISDM dataset.

TABLE III
TOP RESULTS ON WISDM DATASET (RANKED BY SCORE)

Model	Pipeline	Acc.	Train(s)	Inf.(s)	Score
DecisionTree	Mean (DS Only)	0.5947	1.074	0.000	0.5946
DecisionTree	Savgol+Naive (Filt.→DS)	0.5901	1.098	0.003	0.5898
DecisionTree	Butterworth (Filt. Only)	0.5894	1.124	0.003	0.5891
DecisionTree	Raw (Baseline)	0.5862	1.192	0.002	0.5860
TinyCNN	Naive (DS Only)	0.4273	21.235	0.940	0.3333
TinyCNN	Savgol+Naive (Filt.→DS)	0.4237	19.796	0.941	0.3296
TinyCNN	Savitzky–Golay (Filt. Only)	0.4178	21.454	0.987	0.3192
TinyCNN	Raw (Baseline)	0.3974	21.446	1.204	0.2770
LogReg	Savgol+Naive (Filt.→DS)	0.2571	3.098	0.002	0.2568
LogReg	Savitzky–Golay (Filt. Only)	0.2565	2.913	0.002	0.2563
LogReg	Savgol+Mean (Filt.→DS)	0.2559	3.011	0.000	0.2559
LogReg	Raw (Baseline)	0.2498	3.313	0.002	0.2495

WISDM proved to be the most challenging dataset across all models and pipelines. Decision Trees emerged as the best-performing model class, achieving up to 59.47% accuracy with near-zero inference time (0.000s) and a composite score of 0.5946 — only marginally above the raw baseline of 0.5860. This small gain suggests that on highly variable, low-feature datasets, preprocessing provides limited benefit for tree-based models.

TinyCNN showed a more meaningful improvement over its raw baseline (0.3333 vs. 0.2770, a gain of 0.056), though absolute accuracy remained low at 42.73%. Crucially, the raw baseline inference time of 1.204s was reduced to 0.940s with naive downsampling, demonstrating that downsampling provides a tangible latency benefit even when accuracy gains are modest.

Logistic Regression performed poorly across all configurations at approximately 25%, with only marginal improvement over the raw baseline (+0.007). This confirms that WISDM's high inter-subject variability and limited 3-axis feature space present challenges that neither preprocessing nor linear modelling can overcome.

Notably, simpler preprocessing configurations — particularly downsampling only — outperformed complex multi-stage pipelines on WISDM for all three models. This suggests that aggressive filtering suppresses activity-discriminative signal components in datasets with high natural variability and limited feature richness.

D. Cross-Dataset Analysis

Several consistent patterns emerged across all three datasets and are summarized below.

Model Performance: TinyCNN delivered the highest raw accuracy on UCI-HAR and PAMAP2, while Decision Trees outperformed all models on WISDM. This indicates that dataset complexity and feature dimensionality strongly govern optimal model selection for TinyML deployments. No single model dominated universally.

Impact of Preprocessing: Preprocessing consistently improved upon raw baselines across all datasets and models, with the largest gains observed on PAMAP2 (Decision Tree: +0.123) and the smallest on WISDM (Decision Tree: +0.009). This suggests that preprocessing is most beneficial on structured, high-dimensional datasets and least impactful on noisy, low-feature datasets.

Preprocessing Order: Across UCI-HAR and PAMAP2, Downsampling → Filtering pipelines generally outperformed Filtering → Downsampling configurations for TinyCNN and Decision Trees. However, this trend was less consistent on WISDM, suggesting that the benefit of operation ordering is dataset-dependent.

Filter Selection: Savitzky–Golay filtering consistently performed better than or equal to Butterworth filtering for TinyCNN across all datasets, while Butterworth filtering was more beneficial for Logistic Regression and Decision Trees on PAMAP2. This suggests that filter selection should be guided by both model architecture and the signal characteristics of the target dataset.

Inference Latency: Decision Trees maintained near-zero inference times across all datasets and preprocessing pipelines (0.000–0.023s), making them consistently the most deployment-efficient model class. TinyCNN inference times ranged from 0.230s on UCI-HAR to 0.987s on WISDM, driven by the larger input dimensionality of the WISDM dataset.

Composite Score Insights: The composite scoring metric effectively surfaced configurations that balance accuracy and latency in a way that raw accuracy rankings would obscure. On PAMAP2, Logistic Regression configurations ranked above TinyCNN configurations despite lower raw accuracy, correctly reflecting their superior deployment efficiency. This validates the utility of the proposed composite metric as a practical tool for TinyML model and pipeline selection.

V. CONCLUSION

This paper presented a systematic and comprehensive evaluation of preprocessing pipelines for TinyML-based time-series classification across three benchmark Human Activity Recognition datasets — UCI-HAR, PAMAP2, and WISDM. The study examined 13 distinct preprocessing configurations, combining Butterworth filtering, Savitzky–Golay smoothing, naive downsampling, and mean downsampling across five pipeline categories, evaluated on three model classes spanning classical machine learning and lightweight deep learning.

The experimental results yielded several key findings. First, TinyCNN consistently achieved the highest classification accuracy on UCI-HAR (95.44%) and PAMAP2 (94.54%), demonstrating the ability of compact convolutional architectures to

capture discriminative temporal patterns in structured sensor data. However, on the more challenging WISDM dataset, Decision Trees outperformed all other models with a composite score of 0.5946, highlighting that model selection must account for dataset characteristics rather than defaulting to deep learning. Second, preprocessing consistently improved upon raw baselines across all datasets and models, with the largest gains observed on PAMAP2 for Decision Trees (+0.123 in composite score) and the smallest on WISDM, confirming that preprocessing is most impactful on structured, high-dimensional datasets. Third, the ordering of preprocessing operations was found to be meaningful — Downsampling $\rightarrow$ Filtering pipelines generally outperformed Filtering $\rightarrow$ Downsampling on UCI-HAR and PAMAP2, though this trend did not generalize consistently to WISDM. Fourth, Savitzky–Golay filtering proved more suitable than Butterworth filtering for TinyCNN across all datasets, while Butterworth filtering was more beneficial for Logistic Regression and Decision Trees on PAMAP2, underscoring the importance of aligning filter choice with model architecture and signal characteristics. Fifth, the proposed composite scoring metric — which jointly penalizes inference latency and training time relative to accuracy — proved effective at surfacing deployment-optimal configurations that raw accuracy rankings would fail to identify, particularly on PAMAP2 where Logistic Regression outranked TinyCNN in composite score despite lower raw accuracy.

From a practical standpoint, this work offers concrete guidance for TinyML practitioners. For applications requiring maximum accuracy on well-structured, high-dimensional sensor data, TinyCNN with Savitzky–Golay based preprocessing is recommended. For ultra-low-latency deployments on microcontrollers, Decision Trees with lightweight downsampling pipelines offer near-zero inference times with competitive accuracy. For highly variable, low-feature datasets such as WISDM, simpler preprocessing and model choices are preferable, as complex pipelines offer diminishing returns.

Several directions remain open for future work. First, quantization-aware training and model pruning should be explored to further reduce TinyCNN's memory footprint and inference latency for deployment on ARM Cortex-M class microcontrollers. Second, hardware-in-the-loop evaluation on physical edge devices such as Arduino Nano 33 BLE or STM32 microcontrollers would provide more realistic latency and energy consumption measurements than CPU-based profiling. Third, the preprocessing pipeline selection process could be automated through neural architecture search or meta-learning approaches, enabling end-to-end optimization of both the preprocessing strategy and model architecture jointly. Fourth, extending this framework to multi-modal sensor fusion — combining accelerometer, gyroscope, and barometric data — presents a promising avenue for improving classification robustness in real-world wearable deployments. Finally, evaluating additional lightweight architectures such as MobileNet-1D, TCN (Temporal Convolutional Network), and quantized RNNs within the same experimental framework would provide a more complete picture of the accuracy–efficiency trade-off space for TinyML HAR systems.

In summary, this work demonstrates that thoughtful preprocessing pipeline design, guided by dataset characteristics and deployment constraints, is as important as model architecture selection in building practical and efficient TinyML solutions for real-world time-series classification tasks.

ACKNOWLEDGMENT

The author would like to thank *Dr. Rahul Kumar Verma*, *Assistant Professor*, Department of Computer Science and Engineering, Indian Institute of Information Technology Lucknow, for their invaluable guidance, continuous encouragement, and constructive feedback throughout the course of this work. The author also thanks the Indian Institute of Information Technology Lucknow for providing the necessary computational resources and academic environment that made this research possible.

REFERENCES

[1] D. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[2] C. R. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov, D. Patterson, D. Pau, J. Seo, J. Sieracki, U. Thakker, M. Verhelst, and P. Yadav, "Benchmarking TinyML Systems: Challenges and Direction," *arXiv preprint arXiv:2003.04821*, 2020.

[3] A. Bulling, U. Blanke, and B. Schiele, "A tutorial on human activity recognition using body-worn inertial sensors," *ACM Computing Surveys*, vol. 46, no. 3, pp. 1–33, 2014.

[4] N. Ravi, N. Dandekar, P. Mysore, and M. L. Littman, "Activity recognition from accelerometer data," in *Proc. AAAI*, vol. 5, pp. 1541–1546, 2005.

[5] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. European Symposium on Artificial Neural Networks (ESANN)*, 2013.

[6] S. Bhattacharya and N. D. Lane, "From smart to deep: Robust activity recognition on smartwatches using deep learning," in *Proc. IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom)*, pp. 1–6, 2016.

[7] A. Reiss and D. Stricker, "Introducing a new benchmarked dataset for activity monitoring," in *Proc. IEEE International Symposium on Wearable Computers (ISWC)*, pp. 108–109, 2012.

[8] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity recognition using cell phone accelerometers," *ACM SIGKDD Explorations Newsletter*, vol. 12, no. 2, pp. 74–82, 2011.

[9] C. Banbury, C. Zhou, I. Fedorov, R. Matas, U. Thakker, D. Gope, V. J. Reddi, M. Mattina, and P. Whatmough, "MicroNets: Neural network architectures for deploying TinyML applications on commodity microcontrollers," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 517–532, 2021.

[10] S. Butterworth, "On the theory of filter amplifiers," *Wireless Engineer*, vol. 7, no. 6, pp. 536–541, 1930.

[11] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.